MODULE 20 · DOMAIN 3 - DEPLOYMENT, PROVISIONING, AND AUTOMATION

Beyond CloudFormation

CDK, Terraform, Git workflows, Service Catalog, Control Tower and Resource Access Manager.

DOMAIN 3 of 5 EXAM WEIGHT 22% MODULE 20 of 35

admin@sthithapragna.com

None of these tools change what AWS does. They change **who owns the state**, **how intent is expressed**, and **who is allowed to press the button**.

The AWS CDK

- Define infrastructure in a **real programming language**: TypeScript, Python, Java, C#, Go. It **synthesises to CloudFormation**.
- Because the output is CloudFormation, **everything from modules 18 and 19 still applies**: stacks, change sets, drift, StackSets.
- Loops, conditionals, functions and unit tests come from the language, not from a transform.
- The state of record is still the **CloudFormation stack**. CDK is an authoring layer, not a separate engine.

> ON THE EXAM
>
> CDK questions are usually CloudFormation questions wearing different clothes. The deployed artefact is a stack either way.

CDK constructs, in three levels

LEVEL	WHAT IT GIVES YOU
L1, the Cfn* constructs	A one-to-one mapping of CloudFormation resources. Verbose, complete, no defaults
L2, curated constructs	Sensible defaults and helper methods. The level you normally write
L3, patterns	Whole architectures in one object, such as a load-balanced Fargate service

ON THE EXAM

L2 is where the value is. One line grants a Lambda function read access to a bucket, and the CDK writes the IAM policy for you.

The CDK workflow

- **cdk bootstrap** once per account and Region. It creates the **CDKToolkit stack**: an S3 bucket, an ECR repository and IAM roles.
- **cdk synth** produces the CloudFormation template. **cdk diff** compares it with what is deployed.
- **cdk deploy** creates a change set and executes it, so a CDK deployment is a CloudFormation deployment.
- **Forgetting to bootstrap** is the classic first failure. Deployment fails before anything is created.

> ON THE EXAM
>
> cdk diff is the CDK's change set preview. Use it in a pull request exactly as module 19 used change sets in Git sync.

CDK or plain CloudFormation?

CLOUDFORMATION WHEN

The template is the artefact

- Operations teams read and edit it directly
- No build step, no runtime, no dependencies
- Service Catalog and StackSets take templates
- Reviewers see exactly what deploys

CDK WHEN

The infrastructure has logic

- Repetition, conditionals, computed values
- Developers already own the code
- Unit tests over infrastructure
- Reviewers must trust the synthesis

THE TIEBREAKER

The reviewability trade is real. A template is exactly what deploys; CDK code is a program that produces what deploys.

Terraform on AWS

- A third-party tool using its own language, HCL, with an AWS **provider**. Also manages non-AWS resources.
- **terraform plan** previews, **terraform apply** executes. Plan is the change set equivalent.
- Terraform keeps its **own state file** recording what it believes exists. CloudFormation keeps that state for you, in the stack.
- The state file is the whole difference. Lose it, corrupt it, or run two applies at once, and Terraform loses track of your infrastructure.

> ON THE EXAM
>
> "We manage AWS, Azure and a SaaS provider from one workflow" is the strongest Terraform signal. CloudFormation and CDK are AWS-only.

Terraform state on AWS, corrected

- Store state in an **S3 bucket**, versioned and encrypted. That part has not changed.
- **Locking with a DynamoDB table is deprecated.** HashiCorp's own documentation says it will be removed in a future minor version.
- The current method is **S3 native locking**: set **use_lockfile = true**. Terraform writes a **.tflock** object beside the state file.
- Cheaper, simpler, and one fewer resource to manage. Both can be configured together during a migration.

> ON THE EXAM
>
> Nearly every SOA-C03 study guide still says "S3 plus a DynamoDB lock table". That was correct, and it is now the deprecated pattern.

Terraform or CloudFormation?

CLOUDFORMATION WHEN

AWS only, no state to own

- AWS manages state inside the stack
- Native drift detection and rollback
- StackSets for multi-account at scale
- Service Catalog consumes templates directly

TERRAFORM WHEN

Multiple providers

- AWS plus other clouds or SaaS in one workflow
- Existing team skills and modules
- A rich module ecosystem
- You own the state file and its protection

THE TIEBREAKER

New AWS-only work with no existing Terraform: CloudFormation or CDK. Anything spanning providers, or an existing Terraform estate: Terraform.

Git as the operational control

- The repository is the source of truth. Deployments come from a **merged commit**, not from somebody's laptop.
- **Pull request review** is where the safety lives. Module 19's Git sync posts the change set into the PR.
- Run **cfn-lint**, **CloudFormation Guard**, **cdk diff** or **terraform plan** in CI, so problems surface before review.
- Any manual console change becomes **drift**. Git only tells the truth if nobody edits production directly.

> ON THE EXAM
>
> The exam's interest in Git is operational, not technical. Branching strategy is not examinable; deploying only from reviewed, version-controlled code is.

AWS Service Catalog

- A **product** is a CloudFormation template. A **portfolio** groups products and is granted to IAM principals.
- Users launch approved products **without holding the underlying permissions**, which is the entire point.
- A **launch constraint** names the IAM role Service Catalog assumes to provision, so the user never needs it.
- Launched instances are **provisioned products**, tracked per user, and updatable to a newer product version.

> ON THE EXAM
>
> "Developers should self-serve databases without permission to create databases" is Service Catalog with a launch constraint. Nothing else does that.

Service Catalog constraints

CONSTRAINT	WHAT IT ENFORCES
Launch	The IAM role used to provision, so users need no underlying permissions
Template	Restricts parameter values: only these instance sizes, only these Regions
Notification	Sends stack events to an SNS topic
Stack set	Provisions the product across multiple accounts and Regions
Tag update	Whether tags may be changed on a provisioned product

ON THE EXAM

Template constraints are how you offer one product safely. The template allows any instance type; the constraint allows three.

AWS Control Tower

- Sets up and governs a multi-account **landing zone**: a management account, plus **Log Archive** and **Audit** accounts, and a Security OU.
- **Account Factory** provisions new accounts to a standard, with networking and guardrails already applied.
- Built on **Organizations, Config, CloudTrail, IAM Identity Center and Service Catalog**. It orchestrates rather than replaces them.
- **Landing zone drift** occurs when someone changes the underlying resources directly. Control Tower detects and reports it.

> ON THE EXAM
>
> Account Factory is a Service Catalog product. That is why the two services appear together so often.

Control Tower controls, in three kinds

- **Preventive** controls are implemented as **service control policies**. They stop the action from happening at all.
- **Detective** controls are implemented as **AWS Config rules**. They report a resource that is already non-compliant.
- **Proactive** controls are implemented as **CloudFormation Hooks**. They block a non-compliant resource before it is provisioned.
- Controls are drawn from a shared **Control Catalog**, and are applied to organisational units rather than to individual accounts.

> ON THE EXAM
>
> Three mechanisms you already know from three different modules, presented as one governance vocabulary. Preventive stops it, proactive blocks the template, detective finds it afterwards.

AWS Resource Access Manager

- Share resources with **specific accounts, organisational units, or the whole organisation**, without copying anything.
- Commonly shared: **VPC subnets, Transit Gateways, Route 53 Resolver rules, prefix lists, License Manager configurations, Capacity Reservations**.
- With a **shared subnet**, participants launch resources into the owner's VPC. One VPC, many accounts, no peering.
- The owner keeps control: participants **cannot modify or delete the subnet**, and cannot see other participants' resources.

> ON THE EXAM
>
> Sharing a subnet is not the same as granting access to an account. The resource stays owned, and only its use is shared.

RAM or a cross-account role?

RAM WHEN

Share the resource

- Participants use the resource in their own account
- The owner keeps control of its lifecycle
- No credentials to assume, no role chaining
- Subnets, Transit Gateways, Resolver rules

CROSS-ACCOUNT ROLE WHEN

Share the access

- A principal acts inside the other account
- Anything RAM does not support
- Temporary credentials via AssumeRole
- Auditable per session in CloudTrail

THE TIEBREAKER

RAM shares a thing. A role shares an identity. If the requirement is "launch instances in the shared VPC", that is RAM.

How the governance pieces fit

- **Organizations** is the account structure and the policy boundary. Everything else builds on it.
- **Control Tower** establishes and governs the landing zone, and applies controls to OUs.
- **Service Catalog** gives teams safe self-service inside that structure. Account Factory is itself a Service Catalog product.
- **RAM** shares infrastructure across the accounts, so central networking is not duplicated in every one.

> ON THE EXAM
>
> A question naming several of these is usually asking which layer solves the stated problem, not which service is best.

> Development teams need to create RDS databases on demand, but the security team will not grant rds:CreateDBInstance to developers. Every database must use approved instance classes, be encrypted, and be tagged with a cost centre.
>
> The platform team has a CloudFormation template that already does all of this.

What should the platform team implement?

A. A Service Catalog portfolio with the template as a product, a launch constraint, and a template constraint on instance classes

B. Grant developers rds:CreateDBInstance with an IAM condition restricting instance classes

C. A StackSet that deploys the template into each development account

D. A CDK construct published to an internal package repository

◆ PAUSE AND DECIDE

What should the platform team implement?

A Service Catalog with both constraints

The launch constraint means developers never hold the permission, and the template constraint limits instance classes. Self-service without privilege.

B Grant the permission with conditions

Directly contradicts the stated constraint. The security team will not grant it, conditioned or otherwise.

C A StackSet

Pushes the same resources to every account on the platform team's schedule. It is not on-demand self-service.

D A CDK construct

Standardises the code and still requires developers to hold the permissions to deploy it. It solves consistency, not privilege.

THE SIGNAL

The stated refusal to grant the permission is the whole question. Only the launch constraint provisions with a role the user does not have.

> A team is setting up Terraform state for a new AWS project in 2026. A colleague shares a runbook from an older project specifying an S3 bucket for state plus a DynamoDB table with a LockID partition key.
>
> A colleague shares a runbook from an older project. It specifies an S3 bucket for state, plus a DynamoDB table with a LockID partition key. The team wants the currently recommended configuration.

What should the team do?

A Follow the runbook: S3 for state and a DynamoDB table for locking

B Use an S3 bucket for state with use_lockfile = true, and no DynamoDB table

C Store state locally and commit it to Git

D Use CloudFormation instead, since Terraform state is unmanaged

◆ PAUSE AND DECIDE

What should the team do?

A S3 plus DynamoDB

Works, and DynamoDB locking is deprecated in HashiCorp's own documentation and slated for removal. It is a runbook that has aged.

B S3 with use_lockfile

S3 native locking is the current mechanism: a .tflock object beside the state file, one fewer resource, and no deprecated arguments.

C Local state in Git

State contains resource identifiers and often secrets, and committing it gives no locking at all. Two teammates applying at once corrupt it.

D Switch to CloudFormation

A reasonable tool choice in general, and it answers a different question. The team asked how to configure Terraform state correctly.

THE SIGNAL

The same shape as the ARC readiness check and Incident Manager scenarios: an experienced colleague's advice that has quietly gone out of date.

A company runs 60 accounts. Each application account currently has its own VPC, and connectivity between them is provided by a growing mesh of VPC peering connections. Address space is becoming difficult to manage.

A central networking team should own the network, while application teams continue to launch their own instances.

What should the company implement?

A Share subnets from a central VPC using AWS RAM, so application accounts launch into the owner's VPC

B Create a cross-account IAM role in the networking account for each application team

C Replace peering with a Transit Gateway and keep one VPC per account

D Use a StackSet to deploy an identical VPC into every account

◆ PAUSE AND DECIDE

What should the company implement?

A RAM-shared subnets

One VPC owned centrally, 60 accounts launching into it. No peering, one address space, and participants cannot alter the subnets.

B Cross-account roles

Shares an identity, so application teams would work inside the networking account. That is the opposite of teams owning their own instances.

C Transit Gateway

The strongest alternative. It genuinely replaces the peering mesh, and it leaves 60 VPCs and 60 CIDR ranges to manage.

D StackSet-deployed VPCs

Makes 60 VPCs consistent rather than reducing them to one. The address space problem is unchanged.

THE SIGNAL

Two problems are stated: a peering mesh and address space. Transit Gateway fixes the first; shared subnets fix both.

> A governance team must ensure that across every account in the organisation, S3 buckets can never be created without encryption. They want the CloudFormation deployment itself to fail, they want existing non-compliant buckets reported, and they want the control managed centrally per OU rather than per account.
>
> The organisation uses AWS Control Tower.

Which combination of Control Tower controls should they apply?

A. A proactive control for the deployment-time block, and a detective control for existing buckets

B. A preventive control only, since SCPs stop the API call

C. A detective control only, with automatic remediation

D. A proactive control only, since it blocks provisioning

◆ PAUSE AND DECIDE

Which combination of Control Tower controls should they apply?

A Proactive plus detective

Proactive is a CloudFormation Hook, so the deployment fails. Detective is a Config rule, which reports the buckets that already exist.

B Preventive only

An SCP blocks the API call, which is strong. It reports nothing about buckets that already exist, and the requirement asks for that.

C Detective only

Finds and fixes non-compliant buckets after creation. The requirement explicitly wants the deployment to fail.

D Proactive only

Blocks new deployments correctly, and says nothing about the buckets already in the estate.

THE SIGNAL

Two requirements, two control types. The word combination in the question is the hint that one control type will not cover both.

> A platform team maintains infrastructure code for 40 microservices. Each needs an almost identical set of resources, differing in a few parameters. The team wants unit tests over the infrastructure definitions and wants to publish a reusable component their developers consume.
>
> Everything is AWS-only and the team already writes TypeScript.

What should the team use?

A AWS CDK, publishing an L3 construct that developers instantiate

B One CloudFormation template with Fn::ForEach over a list of services

C Terraform modules with an S3 backend

D A Service Catalog product per microservice

◆ PAUSE AND DECIDE

What should the team use?

A CDK with a published L3 construct

Unit tests come from the language, an L3 construct is exactly a reusable component, and the team already writes TypeScript.

B Fn::ForEach in one template

Good for repetition inside one stack. It gives no unit tests, and one template for 40 services means one blast radius.

C Terraform modules

Modules are genuinely reusable and the estate is AWS-only with no existing Terraform, so this adds a state file to own for no gain.

D Service Catalog products

Solves governed self-service, not code reuse. Forty products would each still need a template maintained.

THE SIGNAL

"Unit tests over infrastructure" is the CDK signal. No template language offers them, because a template is data rather than code.

The five things to carry forward

- **CDK synthesises to CloudFormation.** A CDK question is usually a CloudFormation question.
- **Terraform's difference is the state file you own.** S3 native locking, not DynamoDB, is current.
- **A launch constraint provisions with a role the user does not have.** That is Service Catalog's whole purpose.
- **Preventive is an SCP, detective is a Config rule, proactive is a CloudFormation Hook.**
- **RAM shares a thing; a cross-account role shares an identity.** Shared subnets collapse a peering mesh.

> ON THE EXAM
>
> Next: images and containers for operators - EC2 Image Builder, AMIs, ECR, and ECS and EKS basics.